ARTIFICIAL INTELLIGENCE

TOPIC 5.1

Sequence models and recurrent networks

Explanatory study notes

Theory, worked examples and revision questions

What you should be able to do

Distinguish learned weights from changing states; count RNN and LSTM parameters; explain backpropagation through time and gates; perform a small state update; compare recurrence, temporal convolutions and attention; explain teacher forcing.

Based on the teacher PDFs supplied in `Lecture Slides`. The notes explain and reorganize that material. Worked calculations are study examples derived from the slide concepts, unless explicitly identified as a slide exercise. Clarifications of ambiguous or incorrect expressions are marked in context. References use **PDF page numbers**, counted from 1, rather than printed slide numbers.

English edition • 24 October 2026

Contents

1 Sequential data: order and context **3**

2 A vanilla RNN: equations, states and predictions **3**

2.1 Character-level example 4

2.2 Parameter count 4

3 Training through time **4**

3.1 Unrolling and shared gradients 4

3.2 Why gradients vanish or explode 4

3.3 Clipping, truncation and architectural changes 5

4 LSTM: a memory state controlled by gates **5**

4.1 One small update 6

5 LSTM sizes and parameters: the extra-material exercise **6**

6 Worked slide exercise: reading the letter b **6**

7 GRUs, stacked networks and bidirectional processing **7**

8 Temporal convolutions **8**

9 Encoder–decoder models and teacher forcing **8**

9.1 Attention reduces the fixed-context bottleneck 8

10 Revision questions and answers **9**

10.1 Questions 9

10.2 Answers 9

11 Source guide **10**

1 Sequential data: order and context

Teacher slides: 5.1. Sequence Models, pp. 3–6.

A sequence is an ordered collection $(x_1, \ldots, x_T)$. Reordering words can change a sentence's meaning; reversing video frames can change the action. A useful model must account for these relationships rather than treating each item as an unrelated sample.

Different tasks have different input/output arrangements. Sentiment classification maps many words to one label. Video labeling may map one sequence to a label at every time step. Translation maps an input sentence to an output sentence of a possibly different length. Image captioning maps an image representation to a sequence of words.

A recurrent neural network (RNN) processes one element at a time while maintaining a hidden state. The same transition is reused for each step, so the model can process different sequence lengths without introducing a new set of weights for each position.

Variable length is not unique to RNNs

Dense layers require a specified feature dimension, but convolutional architectures can also accept variable spatial or temporal sizes when their readout permits it. The important RNN distinction here is its recurrent state and shared transition, not that every other architecture must have a fixed number of inputs.

2 A vanilla RNN: equations, states and predictions

Teacher slides: 5.1. Sequence Models, pp. 6–12.

Using column vectors, input size D, hidden size H, and output size O:

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h), \qquad s_t = W_{hy}h_t + b_y.$$

The hidden state $h_t \in \mathbb{R}^H$ summarizes the current input and previous state. For classification or next-token prediction, the scores s_t usually pass through softmax to obtain probabilities. Scores themselves need not lie in $[0, 1]$ or sum to one.

Object	Shape	Role
x_t	$D \times 1$	Current observation or token representation.
W_{xh}	$H \times D$	Maps input features into the state update.
W_{hh}	$H \times H$	Maps the previous hidden state into the update.
b_h	$H \times 1$	Hidden bias.
h_t	$H \times 1$	Computed state, different at different steps.
W_{hy}	$O \times H$	Maps the state to output scores.
b_y	$O \times 1$	Output bias.

Parameters are learned; states are computed

During an ordinary forward pass with fixed model weights, x_t, h_t, scores and predictions can change at every step. The shared weight matrices do not change merely because the next token arrives. An optimizer updates parameters during training, after gradients have been computed. A hidden-state entry is a learned feature, not automatically a particular word or

class.

2.1 Character-level example

For a vocabulary of V characters, a one-hot input has $D = V$ and a next-character output has $O = V$. Multiplying W_{xh} by a one-hot vector selects one column of W_{xh}. That column is the input contribution associated with the character; the final state also depends on the preceding context through $W_{hh}h_{t-1}$.

The slide examples show units responding to patterns such as URLs or bracket structure. This illustrates that learned states can track useful context. It does not imply that every unit has a simple human-readable meaning or that the network contains a separate unit for every possible concept.

2.2 Parameter count

With one hidden bias and one output bias,

$$N_{\text{RNN}} = HD + H^2 + H + OH + O.$$

For $V = 3, H = 2$, a next-character model has $6 + 4 + 2 + 6 + 3 = 21$ parameters. Processing 10 rather than 100 characters changes the number of operations and stored training activations, not this count. The slide's displayed vanilla-RNN equations omit biases; if a question omits them explicitly, remove the corresponding terms.

3 Training through time

Teacher slides: 5.1. Sequence Models, pp. 13–16, 32.

3.1 Unrolling and shared gradients

Unroll the recurrence over T steps. This gives a computation graph with T copies of the transition operation, all using the same parameters. A common objective is

$$L = \sum_{t=1}^{T} \ell_t, \qquad \ell_t = -\log p_\theta(y_t^* \mid x_{\le t}).$$

Backpropagation through time (BPTT) applies the chain rule to that unrolled graph. Because the same weight contributes at many time steps, its gradient accumulates contributions from those uses. It is still one trainable parameter.

3.2 Why gradients vanish or explode

A distant state's influence contains a product of Jacobians:

$$\frac{\partial h_t}{\partial h_k} = J_t J_{t-1} \cdots J_{k+1}, \qquad J_t = \operatorname{diag}(1 - h_t^2)\, W_{hh}.$$

Repeated multiplication by factors with small norms can make the gradient tiny. Factors with large amplification can cause it to explode. Saturated tanh units also have small derivatives. For a scalar illustration, ten factors of 0.5 yield $0.5^{10} \approx 0.00098$; ten factors of 2 yield 1024.

The slides' singular-value argument is a bound: a largest singular value below one gives a contraction bound for this vanilla transition. A value above one permits growth; it does not guarantee that every gradient grows, because activation derivatives and directions also matter.

3.3 Clipping, truncation and architectural changes

Norm clipping rescales a gradient vector g when $\|g\| > \tau$:

$$g_{\text{clip}} = g \min\left(1, \frac{\tau}{\|g\|}\right).$$

For $g = (3, 4)$ and threshold $\tau = 2$, the norm is 5 and the clipped vector is $(1.2, 1.6)$. Its direction is unchanged. This limits an exploding gradient; it does not recover a gradient that has vanished. The parameter step also depends on the optimizer.

Truncated BPTT limits differentiation to a window of K steps. A state may be carried forward while its earlier computation history is detached. This saves memory and computation, but cuts the gradients needed to assign credit farther back. LSTM/GRU gates, suitable initialization and normalization address different aspects of recurrent optimization; they are not interchangeable with truncation.

4 LSTM: a memory state controlled by gates

Teacher slides: 5.1. Sequence Models, pp. 17–21; 5.1. Extra Material, pp. 2–4.

An LSTM has both a cell state c_t and a hidden state h_t, each of dimension H. For each gate/candidate $r \in \{f, i, g, o\}$, form

$$a_r = W_{xr} x_t + W_{hr} h_{t-1} + b_r.$$

Then

$$\begin{aligned} f_t &= \sigma(a_f), & i_t &= \sigma(a_i), \\ g_t &= \tanh(a_g), & o_t &= \sigma(a_o), \\ c_t &= f_t \odot c_{t-1} + i_t \odot g_t, & h_t &= o_t \odot \tanh(c_t). \end{aligned}$$

Here $\odot$ is elementwise multiplication. The sigmoid gates lie between 0 and 1; the candidate g_t lies between -1 and 1. The cell state is not restricted to either interval.

Quantity	Interpretation
Forget gate f_t	Fraction of the old cell content retained. Near 1 keeps it; near 0 removes it.
Input gate i_t	How much of the new candidate is written.
Candidate g_t	Proposed signed content, not itself a probability.
Output gate o_t	How much of the transformed cell state is exposed as h_t.

The additive cell update supplies a path that can preserve information. Along the direct cell-to-cell path, the derivative is the forget gate: if it stays near one, that path avoids repeated multiplication by a dense recurrent matrix. Gates help with long dependencies, but do not guarantee unlimited memory or perfect gradients.

4.1 One small update

Suppose one unit has $c_{t-1} = 2$, $f_t = 0.5$, $i_t = 0.25$, $g_t = 0.8$, and $o_t = 1$. Then

$$c_t = 0.5(2) + 0.25(0.8) = 1.2, \qquad h_t = \tanh(1.2).$$

The old contribution is 1 and the newly written contribution is 0.2. Closing the output gate would hide the cell from the output without necessarily erasing the stored cell value.

5 LSTM sizes and parameters: the extra-material exercise

Teacher slides: 5.1. Extra Material, pp. 5–6.

The class exercise uses a one-hot vocabulary $\{a, b, c\}$, so $V = 3$, with $H = 2$. Adopt the column-vector convention above and one combined bias vector per gate.

Object	Shape	Meaning
x_t	3×1	Selects the current letter.
W_{xf}, W_{hf}, b_f	2×3, 2×2, 2	Forget-gate input weights, recurrent weights and bias.
Stacked W_x	8×3	Four blocks, each with two units.
Stacked W_h	8×2	Four recurrent blocks.
Stacked bias	8	Two biases in each of four blocks.
c_t, h_t	2×1 each	Computed memory and exposed state.
W_y, b_y	3×2, 3	Next-character output head.

The LSTM cell has

$$\boxed{N_{\text{LSTM}} = 4(HD + H^2 + H) = 4H(D + H + 1).}$$

For $D = 3, H = 2$, this is $4(6 + 4 + 2) = 48$. The output head adds $3(2 + 1) = 9$, giving 57 in total. Implementations with separate input and recurrent bias vectors count extra stored bias parameters; a counting question must state its convention.

The input-specific contribution of letter b is stored in column b of stacked W_x: eight numbers here. Its contextual effect also depends on recurrent weights and the current state. Its output score has its own row in W_y and output bias; input encoding and output prediction are different roles.

Adding a fourth letter while keeping $H = 2$ expands x_t, W_x and the output head. It does not change the lengths of c_t, h_t or the shape of W_h. The cell now has $4(8 + 4 + 2) = 56$ parameters. Conversely, keeping $V = 3$ and doubling H to 4 gives $4(12 + 16 + 4) = 128$ cell parameters, plus 15 output-head parameters. Doubling hidden width is more than a doubling of the recurrent parameter count because of the H^2 term.

6 Worked slide exercise: reading the letter b

Teacher slides: 5.1. Extra Material, p. 7.

This is the teacher's numerical exercise, using the supplied rounded sigmoid and tanh values. Start with $c_{t-1} = (1, 0.5)$, $h_{t-1} = (0.5, 0)$ and $x_t = (0, 1, 0)$. The one-hot input selects the second column of each input matrix.

Applying the displayed weights gives:

$$\begin{aligned} a_f &= (2 + 4(0.5),\ -4 + 2(0)) = (4, -4), \\ a_i &= (-4 + 0,\ 4 + 0) = (-4, 4), \\ a_g &= (0,\ 2 - 2(0.5)) = (0, 1), \\ a_o &= (0 + 3(0),\ 0 + 2(0.5) - 1) = (0, 0). \end{aligned}$$

The gates and candidate are therefore approximately

$$f = (0.98, 0.02), \quad i = (0.02, 0.98), \quad g = (0, 0.76), \quad o = (0.5, 0.5).$$

Update the cell elementwise:

$$c_t = (0.98(1) + 0.02(0),\ 0.02(0.5) + 0.98(0.76)) = (0.98, 0.7548).$$

Using the slide's approximate $\tanh(0.98) = 0.75$ and rounding 0.7548 to about 0.76, with $\tanh(0.76) \approx 0.64$,

$$h_t \approx (0.375, 0.32).$$

The first unit mostly keeps its old cell content and writes nothing; the second mostly erases its old content and writes the new positive candidate. Both expose about half the transformed cell content.

For the bonus head $W_y = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ -1 & -1 \end{bmatrix}$ and zero bias, scores are approximately $(0.375, 0.32, -0.695)$. Greedy prediction chooses a. These numbers are logits, not probabilities; softmax would preserve their ordering. Exact nonlinearities give slightly different decimals because the supplied lookup values are rounded.

7 GRUs, stacked networks and bidirectional processing

Teacher slides: 5.1. Sequence Models, pp. 21–24.

A gated recurrent unit (GRU) uses update and reset gates, with a candidate state, but no separate LSTM-style cell state. It interpolates between an old state and a proposed state. Different notations swap the roles of z and $1 - z$, so interpret the stated update equation before saying what a gate value near one means.

With three affine blocks and one bias per block, the cell count is $3H(D + H + 1)$, compared with $4H(D + H + 1)$ for the LSTM convention above. Fewer parameters does not guarantee better accuracy; it is an architectural trade-off.

Stacking recurrent layers adds depth: the lower layer's output at time t becomes an input to a higher layer. Each layer normally has its own weights. A bidirectional RNN processes a sequence in both directions and combines the resulting features. This gives access to future context within the observed input, but it cannot be used unchanged for online prediction when those future observations are unavailable.

For video, a CNN can encode individual frames and a recurrent model combine their temporal features. For captioning, an image or region representation conditions a word decoder. The task determines whether one final state, every state, or an output sequence is required.

8 Temporal convolutions

Teacher slides: 5.1. Sequence Models, pp. 25–27.

A temporal convolutional network (TCN) applies convolutions along time. A causal layer at position t uses only positions at or before t; the target alignment determines whether x_t itself is already known. A centered filter using future observations would leak information in a forecasting task.

Dilation increases the temporal span without increasing the number of filter coefficients. For stride-one layers of kernel width K and dilations $d_1, \dots, d_L$, a simple sequential stack has receptive field

$$R = 1 + (K-1)\sum_{\ell=1}^{L} d_\ell.$$

Two layers with $K = 3$, dilations 1 and 2, cover $1 + 2(1 + 2) = 7$ time positions. WaveNet illustrates stacked dilated causal convolutions. Training positions can be processed in parallel when all inputs are known, but autoregressive generation still depends on previously generated samples.

9 Encoder–decoder models and teacher forcing

Teacher slides: 5.1. Sequence Models, pp. 28–31.

An encoder reads the input sequence and produces a context representation. A decoder generates the target sequence, usually beginning with a start token and stopping at an end token. Input and output lengths need not be equal.

In basic recurrent seq2seq, the encoder's final state is a fixed-size context vector. This can become a bottleneck: information from a long input must be compressed into that one vector.

During **teacher forcing**, the decoder receives the true previous target token while learning to predict the next one:

$$L = -\sum_t \log p_\theta(y_t^* \mid y_{<t}^*, x).$$

At generation time, the true target is unavailable, so generated tokens are fed back. An early error can create contexts less familiar from training: exposure bias. Scheduled sampling mixes in model-generated inputs during training; it is an attempted remedy, not a guarantee.

Teacher forcing does not remove recurrence from an RNN decoder: h_t still depends on h_{t-1}. A nonrecurrent transformer decoder can instead compute training positions in parallel under a causal mask.

9.1 Attention reduces the fixed-context bottleneck

Keep all encoder states and form a different context for each output step:

$$c_t = \sum_{i=1}^{T} \alpha_{t,i} h_i, \qquad \alpha_{t,i} \geq 0, \quad \sum_i \alpha_{t,i} = 1.$$

The weights depend on the current decoding query and the encoded inputs. If two scalar values are 2 and 6, weights $(0.75, 0.25)$ produce context 3. Different output tokens can attend to different source positions. Attention can be added to an RNN; a transformer goes further by building its main sequence processing from attention blocks.

10 Revision questions and answers

10.1 Questions

1. Which changes during a forward pass: the recurrent weights or the hidden state?
2. How many parameters has an RNN with $D = 3, H = 2, O = 3$, including both biases?
3. Why does an unrolled RNN have more computation but not more parameters when T increases?
4. What does BPTT do with gradient contributions from repeated uses of a shared weight?
5. Does clipping fix vanishing gradients? Does truncated BPTT preserve arbitrarily long credit assignment?
6. What do $f_t \approx 1$ and $i_t \approx 0$ do to the LSTM cell?
7. Compute c_t for $c_{t-1} = 2, f_t = 0.5, i_t = 0.25, g_t = 0.8$.
8. How many cell parameters has an LSTM with $D = 3, H = 2$ and one bias per gate?
9. Does adding one vocabulary symbol necessarily increase the hidden-state dimension?
10. Why is a bidirectional RNN unsuitable for strict online forecasting?
11. What does teacher forcing supply to the decoder? What changes at inference?
12. What does attention change in a basic fixed-context seq2seq model?

10.2 Answers

1. The hidden state; weights are shared and fixed within the ordinary forward pass.
2. $6 + 4 + 2 + 6 + 3 = 21$.
3. More applications of the same transition are computed.
4. They are accumulated into the gradient of that one shared parameter.
5. No to both: clipping caps large gradients; truncation cuts long gradient paths.
6. Mostly preserves the old cell and writes little new content.
7. $1 + 0.2 = 1.2$.
8. $4(6 + 4 + 2) = 48$, excluding an output head.
9. No; input/output vocabulary dimensions change, while hidden width can stay fixed.
10. Its reverse-direction state depends on future observations.
11. The true previous token during training; generated tokens during inference.
12. Each output can access a weighted combination of all encoder states instead of relying solely on one fixed context.

11 Source guide

`5.1. Sequence Models.pdf`: sequence structure and vanilla RNNs (pp. 3–12), BPTT and gradient problems (pp. 13–16), LSTM/GRU and variants (pp. 17–24), TCNs (pp. 25–27), seq2seq and attention (pp. 28–32), and the character-model exercise (p. 33).

`5.1. Extra Material.pdf`: cell diagrams (pp. 2–4), sizes/counts (p. 5), parameters versus states (p. 6), and the worked numerical update (p. 7). These notes incorporate this extra deck into 5.1 rather than treating it as a different course topic.